

Game Design Document

A Worked Example - Semester 1 Capstone

Every section of your own Game Design Document Template, filled out completely

VIDEO GAME CREATION

Student Name:
(worked example - this is what a completed notebook looks like, not a real student's)

Game Title:
Kestrel Watch

Engine / Platform:
Written engine-agnostic on purpose - every design choice below applies the same way whether you built this in Scratch, Godot, Unity, GameMaker, or any of the other 8 supported engines. Your own doc should name YOUR engine.

Started (Lesson #/Date):
Lesson 2

How to Use This Document

This is the exact same template as your own Game Design Document - same sections, same tables, same prompts - filled out completely, the way it would look if you'd kept it faithfully all the way through Lesson 15. It's not meant to be read as "the right answers" (your game will be your own, especially once Semester 2 starts). It's meant to show what GOOD, THOROUGH thinking looks like in each section - reasoning, not just a list of features.

- Notice how often an entry explains WHY a choice was made, not just WHAT the choice was. "We used lane-based descent" is a fact. "We used lane-based descent because formation marching needs shared list state we hadn't built yet, and we didn't want Level 1 blocked on a Level 3 idea" is a decision you can actually learn from.
- Your own document doesn't need to be this long, especially early on. Short, honest, and actually written down beats long and never updated - revisit it, don't try to perfect it in one sitting.
- Sections marked "WORKED EXAMPLE NOTES" (in place of your template's "YOUR NOTES") call out the actual reasoning behind a real decision this class made, so you can see the thought process, not just the conclusion.

The Big Picture

Genre

Examples: platformer, top-down adventure, puzzle, shooter, racing, simulation...

Top-down arcade shooter, in the tradition of Space Invaders and Galaga.

One-Sentence Pitch

Example: "A robot has to reach the top of a crumbling tower before it collapses."

A lone pilot holds a frontier relay station against wave after wave of an unknown signal-drawn swarm, each wave faster and larger than the last.

Games it's inspired by

What games (or movies, books, etc.) gave you this idea?

Space Invaders and Galaga specifically - chosen for minimal art/animation requirements and a core loop that scales cleanly from a one-lesson MVP all the way to a full semester of features, without ever needing to be rebuilt from scratch.

Core Gameplay

Lesson 2's Key Press Reactor is this question in its simplest possible form: what happens when you press a key? Everything added later built on that same idea.

Core Mechanic

What's the ONE thing the player does over and over? Everything else in your game supports this one action.

Move along the bottom of the screen and shoot upward to destroy enemies before they reach you. Every later feature (waves, lives, score, sound, difficulty scaling) exists to make that one action matter more over time, not to replace it.

Controls

Started as Lesson 2's Key Press Reactor mapping and grew one row at a time as new mechanics arrived - nothing about the original left/right mapping ever had to change.

Input	Action	Still accurate?
Left Arrow	Move left along the bottom of the play area	Yes - unchanged since Lesson 2
Right Arrow	Move right along the bottom of the play area	Yes - unchanged since Lesson 2
Spacebar	Fire a shot upward	Added Lesson 3 - still accurate
R (when game is over)	Restart the run	Added Lesson 10 - still accurate

Win Condition

How does the player succeed?

Clear every enemy in the current wave. At Level 1 that's a single wave; from Level 2 onward (Lesson 11+) it's every wave in a run, with a real "you win" state once the last wave clears.

Lose Condition

How does the player fail? (Not every game needs one!)

An enemy reaches the player's row, or the player runs out of lives (Level 2+, 3 lives by default). Losing a life plays a sound and keeps the run going; losing the last life ends it.

World & Characters

This section is easy to skip or leave thin - don't. A few named, specific choices here made a lot of LATER decisions easier, because we could ask "does this fit the story we already told ourselves?" instead of guessing from scratch every time. It doesn't need to be great writing. It needs to exist.

Player Character

Who / what is the player controlling?

A single pilot, callsign Vega, flying a small single-seat defense craft called the Lance. Vega is stationed at Relay Station Kestrel, a lightly-staffed listening post at the edge of mapped space - not a hero ship from a big fleet, just whoever happened to be on watch when the signal came in. Keeping the player deliberately ordinary (not a chosen-one, not a flagship) matched the tone we wanted: calm, a little lonely, arcade-y rather than epic.

Setting / World

Where does this take place?

The Halcyon Verge - a quiet frontier sector, mostly empty space, with Relay Station Kestrel as its only real outpost. Kestrel's job before this game starts was boring: listen for anything unusual coming in from deeper space and report it. The game begins the moment that job stops being boring.

Other Characters, Enemies, or NPCs

The Drift - not a traditional alien empire with a face and a plan, but a self-replicating probe swarm drawn toward Kestrel's own listening signal. Individual units are called driftlings - small, numerous, and disposable from the swarm's point of view, which is exactly why there are more of them every wave rather than a few tougher ones.

Each wave is a new brood, recalibrated based on what the last brood learned before it was destroyed - this is the in-story reason enemy count AND speed both increase wave over wave (Lesson 14), rather than the game just getting arbitrarily harder for no reason. It also gave us a natural explanation for the Level 3 stretch feature the live class actually built: a formation-marching brood (moving as a group, dropping a row at each edge, instead of each driftling descending independently) represents a more coordinated, more evolved wave type that shows up once a player has pushed the Drift far enough to provoke one.

The other four Level 3 stretch options, added later in the consolidated reference build, fit the same story without needing a rewrite: a 'lead driftling' bonus enemy (worth more points, moving a bit faster, every other brood) represents an early scout taking point; the destructible shields are Kestrel's own point-defense barriers, not a Drift feature at all; the power-up is a brief burst of salvaged signal-boost tech from a destroyed driftling, explaining why it temporarily speeds up Vega's own fire rate; and the title screen framing - 'Kestrel Watch,' press SPACE to begin your watch - is literally the listening-post duty the station existed for before any of this started.

Levels / Structure

Reminder - how "Level 1 / 2 / 3" works in this class:

Level	Think of it as...	What it means
Level 1 - Minimal	MVP	The smallest working version - enough to understand the idea and move on. A complete, legitimate finish.

Level	Think of it as...	What it means
Level 2 - Complete	Internal Alpha/Beta	All expected features present and working end-to-end. Art/polish/balance may still be missing.
Level 3 - Refined	Ready to Publish	Feature-complete, tested, and polished enough to comfortably share outside of class.

This game doesn't have traditional levels/areas - it has waves. We still used this table, reframed around the story above, to describe how the SAME mechanical progression (more enemies, faster enemies) reads differently depending on how far into a run a player has gotten:

Wave Tier (narrative framing)	What actually happens there	Status
Outer Watch (Waves 1-2)	A small brood of 5-6 driftlings probes the relay's defenses. Slow, predictable, lets a player learn the controls without punishment.	Done
Mid Verge (Waves 3-5ish)	Larger broods (the 4 + wave formula), moving measurably faster each wave - the Drift is recalibrating after each failed probe.	Done
Core Approach (later waves)	Where our formation-marching brood shows up - a visibly different, more organized threat late in a run. This was the one Level 3 stretch feature the live class picked and built; the capstone's Complete Codebase reference build later added the other four (shields, a power-up, a bonus enemy, and the Kestrel Watch title/game-over screen) on top of it, so every stretch option has a working example somewhere in the final project.	Done - all five stretch options present in the reference build

Visual Art Plan

Placeholder art is completely fine while you're prototyping - the lesson Assets folders always include ready-to-use placeholder sprites so you never have to stop and hunt for art mid-lesson. Use this section to describe what you eventually want your game to look like.

Art Style

Examples: pixel art, flat cartoon, simple geometric shapes, hand-drawn...

Simple, flat, high-contrast shapes - the provided placeholder set (player ship/arrow sprite, a uniform invader silhouette, a basic explosion burst) was never swapped out. For a project this size, clarity at a glance mattered more than visual polish - every enemy reads instantly as "enemy," which matters once a wave has 7+ of them on screen moving at once.

Color Palette / Mood

Cool, calm background tones (space, a quiet frontier) with a warm, high-contrast red/orange enemy silhouette and a true-white hit flash, so the one moment that most needs to read instantly - a hit landing - is also the moment with the most visual contrast.

Placeholder assets currently in use (check any that apply)

- ☐ player_circle_blue.png / player_circle_green.png / player_circle_orange.png
- ☑ player_arrow_white.png
- ☐ player_walk_spritesheet.png
- ☐ My own art (describe below)

Notes on your own art assets

None this semester - every asset used (player, enemy, bullet, explosion, the Lesson 13 white hit-flash variant) came from the course's own Assets folders. Custom art is explicitly Semester 2 scope, not a Semester 1 expectation.

Audio Plan

sfx_blip.wav is the placeholder sound effect provided in early lessons - swap it out here with what you actually want once you know.

Music Style / Mood

No music track this semester - four short sound effects carried the whole game, and a 15-lesson arcade shooter didn't need a music budget to feel complete. Worth reconsidering for Semester 2's bigger scope.

Sound Effects Needed

List the moments in your game that should make a sound.

Firing a shot (Lesson 3's shoot action)

An enemy getting hit (Lesson 13's flash moment)

An enemy destroyed / wave cleared (Lesson 11's wave-advance)

Losing a life (Lesson 8's lose-condition)

Game over (final loss)

Scope & Milestones

This is the class's actual 30-lesson schedule (see the Course Syllabus for full detail). This worked example only covers through Lesson 15 on purpose - Semester 2 starts a brand-new, fully your-own-design game, so there's nothing from this project to show you there. Your own document keeps going where this one stops.

Lessons	Class Focus	Class Milestone	Status (this example)
1	Orientation: internet safety, game dev overview, platform selection	Engine installed & first project opened	Done
2-6	Programming foundations: variables, events, loops, conditionals, input	First moving, controllable character	Done
7-11	Core mechanics: collision, scoring, win/lose states, simple UI	Small playable prototype	Done
12-15	Game feel & polish: audio, basic animation, level structure	Semester 1 capstone: complete Mini-Game	Done - this document
16-20	Intermediate systems: state machines, enemy/AI behavior, physics	Prototype with opponents/obstacles	Semester 2 - not this example
21-25	Expanding scope: menus, save data, multiple levels, juice/polish	Feature-complete draft of capstone game	Semester 2 - not this example
26-28	Playtesting, bug-fixing, presentation prep	Playtested, stable build	Semester 2 - not this example
29-30	Finalize, package, and export shareable copies	Shareable build ready for CoOp Expo	Semester 2 - not this example

Semester 1 Capstone (Lessons 12-15)

By the end of Lesson 15, the game was a complete Mini-Game: a clear core mechanic, art and audio (placeholder, and that was fine), and a real win or lose state. This is the same project that started in Lesson 2 - it grew the whole time, it was never restarted.

My plan for reaching the Semester 1 capstone

Stayed at Level 1 through Lesson 6 rather than rushing ahead - a complete, working MVP beat a half-finished Level 2 attempt. Treated Lessons 12-14's new features (sound, hit flash, difficulty scaling) as genuinely separate additions, each tested on its own before moving to the next, rather than changing three things at once and guessing which change broke something when a bug showed up.

Semester 2 / CoOp Expo Capstone (Lessons 26-30)

Not applicable to this worked example - this document stops at the Semester 1 capstone on purpose, since Semester 2 is a new, individually-designed game with nothing from this project to carry over design-wise. The habits above (stay at Level 1 until it's solid, test one new thing at a time, keep this journal honest) are exactly what DOES carry over, even though the game itself doesn't.

The Five Disciplines of Game Development

Every game (including yours) touches all five of these, even on a small student project. Programming is the class's main focus, but the other four matter too - here's what each one actually looked like on this project.

Design

Deciding what the game is and how it plays - the rules, the goals, the feel.

WORKED EXAMPLE NOTES

Most of our actual design decisions this semester were really scope decisions: rather than attempting shields, a power-up, a bonus enemy, formation marching, AND a title screen all at once in the live class, we picked exactly one (formation marching) once the core loop was solid, and left the other four as ideas for later rather than four half-built features. A smaller, finished idea beats a bigger, half-built one. Afterward, once code-complete, we went back and actually built out the other four as part of a single consolidated reference example - at that point it was no longer about keeping a live class's weekly workload small, it was about having one complete, fully-reconciled example to point every future student at, regardless of which single stretch feature their own class session chooses.

Programming

Making the rules and mechanics actually work in code (our main focus this year).

WORKED EXAMPLE NOTES

The single biggest programming lesson of the semester was the Stalemate bug (Lesson 10) - two code paths (a kill and an escape) that each thought they were the only way a wave could end. Any time two different events can both lead to the same outcome, check that BOTH of them actually update the SAME state.

Visual Art

Characters, environments, and UI - what the player sees.

WORKED EXAMPLE NOTES

We stayed on placeholder/provided art the entire semester - a circle for the player, a simple invader silhouette for enemies, a basic explosion sprite - and that was completely fine. The game reads clearly with programmer art; custom art is Semester 2 scope, not Semester 1.

Audio

Music and sound effects - what the player hears.

WORKED EXAMPLE NOTES

Sound didn't show up until Lesson 12, on purpose - adding it before the core loop was solid would have meant re-doing sound cues every time a mechanic changed. Four short sound effects (shoot, hit, game over, wave clear) did almost all the work; no music was needed for a project this size.

Production

Planning, scoping, and finishing a project - relevant all year, critical before the Expo.

WORKED EXAMPLE NOTES

The Level 1/2/3 framework WAS our production plan: Level 1 is a real, shippable finish on its own, so falling behind never meant falling permanently behind - it just meant staying at Level 1

a little longer. That's the single habit most worth carrying into Semester 2, where there's no shared schedule to lean on.

Design Journal / Change Log

A real running log, one entry per lesson where something meaningful changed and why - this is the single most useful section in this whole document to actually keep up, and the easiest one to let slide. These are genuine entries pulled from this semester's actual production notes, not invented after the fact.

Lesson #	What changed and why
2	Direction-color feedback: planned to tint the ship's sprite per direction, but on our colored placeholder art the engines' quick-tint properties multiply into a muddy color instead of a clean one. Switched to sprite-swap (a separate pre-made sprite per direction) - simpler to explain, needs no extra assets, and works identically in every engine.
5	Enemy movement: considered classic formation marching (the whole wave moves together, drops a row at each edge) for the very first multi-enemy pass, but it needs shared list/array state that's a stretch for this point in the course. Went with lane-based descent (each enemy moves independently) for Level 1/2, and kept formation marching as an optional Level 3 stretch goal once lists are comfortable.
10	Found a real bug during the win/lose work: a round could end with no win and no loss, because kills reduced enemies_remaining but escapes only reduced lives - so an escaped-to-zero wave never registered as cleared. Fixed by having escapes ALSO reduce enemies_remaining, and named the mistake clearly (we call it "the Stalemate" now) so we'd recognize the shape of the bug if it ever came back in a different form.
11	Multiple waves introduced wave and total_waves as new shared state. Decided both belong on the same GameManager-style object that already owned score/lives, rather than scattering wave logic into whichever script happened to trigger it - keeping ALL shared game state in one place made every later lesson easier to reason about.
12	Added sound for the first time: shoot, hit, explosion, game over, wave-clear. Decided sound effects are optional/stretch-feeling right up until this lesson, then became part of the Level 3 baseline everyone is expected to reach - sound is a genuinely different kind of feature from a mechanic, and it deserved its own dedicated lesson rather than being bolted onto whichever lesson happened to need a sound cue.
13	Added the hit flash and explosion. Learned the hard way that a quick white-tint flash doesn't actually work in most of these engines - a "color effect"/tint property shifts or multiplies color rather than blending to true white. Fixed by swapping to a real white-painted sprite instead of trying to fake white with a tint.
14	Added wave-based difficulty scaling (enemy count AND speed both grow with the wave number), using the exact same wave variable that's existed since Lesson 11 - a good example of a later feature paying off a piece of state that was already sitting there, unused, for three lessons.
15	Capstone wrap-up: confirmed restarting correctly resets wave, count, and speed back to wave 1's values (not wherever the last run left off), and picked ONE Level 3 stretch feature (the formation-marching brood described in World & Characters above) for the live class to build, rather than trying to do all five at once mid-course.
Capstone Codebase	After the semester wrapped, we assembled one consolidated "Complete Codebase" reference build - partly so students who want to see a complete example aren't limited to just the one stretch feature their own class session happened to build. That build implements ALL FIVE of Lesson 14's Level 3 stretch options together (shields, a power-up, a bonus enemy, formation marching, and a title/game-over screen), reconciled so they don't conflict with each other (formation waves and the bonus enemy's tint, for instance, both needed to apply to the SAME enemy without one silently overwriting the other). It also restored several earlier-lesson Level 3 "pick any that interest you" options that a typical live class session skips in the normal course of picking a couple

Lesson #	What changed and why
	favorites rather than all of them: Lesson 4's distinct enemy color, Lesson 5's striped-row coloring, Lesson 6's centered on-screen text, Lesson 7's per-enemy speed variation, Lesson 9's scoring variety, and Lesson 10's fade-in win/lose text. None of this changes what Level 1 or Level 2 require - it only means the Level 3 "ready to publish" ceiling is now fully built out in one place, for anyone who wants to see every option working at once.

Looking Back: What We'd Do Differently

A short, honest retrospective - the kind every finished project deserves, even a small one. Being specific here is more useful than being complimentary.

- The Stalemate bug (Lesson 10) existed because two separate places both changed enemies_remaining-adjacent state without agreeing on the rule. If shared state had been consolidated onto one object from Lesson 6 instead of Lesson 10, that bug likely never would have happened at all - a good argument for deciding EARLY where important numbers live, not just when a bug forces the question.
- Enemy spawn positions were hand-picked per engine back in Lesson 5 and never revisited when Lesson 14 made enemy count variable - the spawn line quietly drifts rather than staying centered as more enemies are added. A named formula (a center point, a per-enemy spacing value, and a wave-based crowding adjustment) would have cost little extra effort up front and saved a "why does this look slightly off" moment later. Worth planning for formulas like this BEFORE a value becomes variable, not after.
- Sound wasn't added until Lesson 12, which was the right call pedagogically, but on a real project you might feel differently about play-testing a long stretch without any audio feedback at all - worth deciding early how long you're comfortable prototyping silently.

What's Different When It's Your Own Design

Everything above describes a shared class project - the whole point of Semester 1 was that nobody had to invent the game AND learn to program it at the same time. Semester 2 removes that scaffolding: the game, the story, the art direction, all of it is yours to decide. A few things worth carrying forward on purpose:

- This document will be much shorter at first, and that's correct, not a shortcut - "Genre: top-down shooter" and a two-sentence pitch is a completely legitimate Lesson 16 entry. The goal is that it exists and gets revisited, not that it's long.
- Give your player, your antagonist, and your setting real names and a one-paragraph reason they're in conflict, even if (especially if) the mechanics come first. It costs almost nothing to write, and it pays for itself every time you're stuck deciding what a new feature should look or feel like - ask what the story would do, the same way "why does the Drift get faster each wave" had an actual answer here instead of just "because it's more fun."
- Keep the Design Journal honest from Lesson 16 onward. You won't have a class-wide milestone schedule telling you when something should be done - this journal is how you'll know, months from now, why you made the choices you made.